

Trading Logic Event-Action Table

0. 책임 및 실행 계약

0.1 TradingSTM과 TradingController의 경계

이 문서의 상태, EVENT, Guard, 판정 우선순위와 다음 상태는 **TradingSTM**의 결정 책임이다. 각 표의 **TradingController** 수행 **Action** 열은 **TradingSTM**이 직접 실행하는 동작이 아니라 **TradingSTMResult.action_requests**로 반환하는 실행 요청이며, 실제 변경과 외부 효과는 모두 **TradingController**가 수행한다.

처리 순서는 다음과 같다.

- 1. **TradingController**가 시장-사용자-주문 결과 EVENT와 동일 평가 시점의 읽기 전용 **TradingContextView**를 **TradingSTM**에 전달한다.
- 2. **TradingSTM**이 현재 상태에서 Guard와 우선순위를 평가하고 transition ID, 다음 상태, 순서가 있는 Action 요청을 **TradingSTMResult**로 반환한다.
- 3. **TradingController**가 표의 Action을 순서대로 수행한다. Context 변경은 **TradingContext**의 typed method를 통해 적용하고, 주문-저장-인계는 해당 Entity/Gateway/Controller에 위임한다.
- 4. Action 수행 결과로 필요한 후속 EVENT를 **TradingController**가 직렬 event queue에 넣는다. STM은 event queue, timer, Gateway를 직접 호출하거나 자기 자신을 재귀 호출하지 않는다.

상태 전이 자체는 STM의 책임이므로 Action 열에 반복해서 적지 않는다. **None** 또는 **없음**은 Controller가 수행할 Action이 없다는 뜻이다.

0.2 Action 요청 표기

Action 요청	TradingController의 수행 책임
PatchRuntimeContext	pause/consumed/recovery flag, signal/flush/timer 값, pending/exit 값 등을 변경한다.
OpenLowerEvent / CloseLowerEvent	하단 터치 snapshot과 event-local Context를 생성·종료한다.
QueueEvent	지정 EVENT를 현재 microstep 종료 뒤 내부 queue에 넣어, 이미 대기 중인 후속 외부 market EVENT보다 먼저 처리한다.
ScheduleReevaluation	시장 값 변경, candle close 또는 deadline에서만 재평가 EVENT를 넣는다. 즉시 busy loop를 만들지 않는다.
SubmitOrder / ForceSellAll	주문 의도를 고정하고 제출·조회·체결 반영·저장을 조정한다.
ReconcileOrder	상태 불명·부분 체결·중지 중 pending 주문을 같은 주문 ID로 조회하고 실제 fill과 잔여 수량을 일치시킨다.
CancelPendingOrder	경쟁 전략, 중지 또는 안전 종료로 무효가 된 취소 가능 주문을 취소한다.
CancelScheduledEvaluation	전략 또는 session 범위의 예약된 재평가를 취소해 종료 중 신규 Action을 차단한다.
StopTradingRuntime	신규 event 수신, timer, 구독과 runtime을 안전하게 종료한다.

각 Action cell의 문장은 위 typed 요청의 payload와 실행 순서를 구체화한다. STM 결과에 Python callable, mutable Entity 또는 Gateway 객체를 담지 않는다.

0.3 주문 요청과 결과 EVENT

주문 Guard와 주문 실행을 같은 transition에서 동시에 평가하지 않는다. 주문은 항상 아래 두 단계로 처리한다.

- 1. STM이 전략 Guard를 만족한 행에서 **SubmitOrder** 또는 **ForceSellAll** Action 요청을 반환한다. 이때 성공 상태로 미리 전이하지 않는다.
- 2. Controller가 주문을 수행한 뒤 정상 체결, 확정 실패, 상태 불명 중 하나로 정규화한다. 정상 체결은 Position·이력·Context 반영 후 완료 EVENT를 발생시키고, 확정 실패는 실패 EVENT를 발생시킨다. **NEW**, **PARTIALLY_FILLED**, **UNKNOWN**은 새 주문을 제출하지 않고 같은 주문 ID를 먼저 조회·조정한다.

주문 종류	정상 체결 후 Controller EVENT	확정 실패 후 Controller EVENT
Case B 매수	CASE_B_POSITION_OPENED	CASE_B_BUY_FAILED(attempt_kind)
Case C 매수	CASE_C_POSITION_OPENED	CASE_C_BUY_FAILED(attempt_kind)
Case B 매도	CASE_B_SELL_FILLED	CASE_B_SELL_FAILED
Case C 매도	CASE_C_SELL_FILLED	CASE_C_SELL_FAILED
중지 전량 매도	FORCE_SELL_FINISHED	FORCE_SELL_FAILED

position_owner는 terminal 주문 결과의 실제 매수 체결 수량을 Position에 반영한 뒤에만 설정한다. 주문 진행 중에는 **pending_strategy**, **pending_order_side**, **pending_order_id**, **trading_phase**로 주문 자리를 예약한다. pending 주문이 있으면 다른 신규 주문 Action을 만들지 않는다.

모든 **SubmitOrder** 실행 전 Controller는 strategy, side, client/order ID와 주문 phase를 Context에 설정한다. 진입 주문의 “pending 값 해제”는 해당 필드를 **None**으로 만들고 **trading_phase = IDLE**로 되돌리는 것을 뜻한다. 청산 주문은 terminal 실패나 부분 체결 뒤에도 **pending_strategy**, **pending_exit_reason**, **pending_return_state**, **trading_phase = EXIT_ORDER_PENDING**을 유지하며, PB-23F/PC-23F에서 완전 청산 피드백을 처리한 뒤 해제한다. STOPPING과 reconciliation 중에는 각 전용 phase를 유지한다.

주문 결과를 반영하는 Controller의 공통 순서는 다음과 같다.

- **매수 정상 체결**: 실제 fill을 Position과 거래 이력에 반영한 뒤 **position_owner**를 해당 Case로 설정하고 주문 pending 값을 해제한 다음 **CASE_*_POSITION_OPENED**를 발생시킨다.
- **매도 정상 체결**: 실제 fill과 원가를 Position·거래 이력·성과에 반영하고 **position_owner = None**으로 변경한다. **case_b_exit_reason** 또는 **case_c_exit_reason**을 **pending_exit_reason**으로 확정하고 Case C는 체결 시점 **case_c_exit_pct_b**도 저장한다. 주문 ID/side/phase는 해제하되 **pending_exit_reason**과 **pending_return_state**는 **CASE_*_SELL_FILLED** 처리까지 유지한다.

- **terminal** 미체결: 주문 ID/side를 해제하되 청산 주문이면 `pending_exit_reason`과 `pending_return_state`를 유지하고 `CASE_*_SELL_FAILED`를 발생시킨다.
- **상태 불명 또는 부분 체결**: 같은 주문 ID를 조회하고 실제 fill을 먼저 reconciliation한다. 중복될 수 있는 신규 주문을 제출하지 않는다.

terminal 상태에서 실제 체결 수량이 0보다 크면 단순 실패로 retry하지 않는다. 실제 체결량을 Position과 이력에 먼저 반영한다. 매수는 실제 수량으로 해당 Case 포지션을 개설한다. 매도 후 잔여 Position이 있으면 `position_owner`와 현재 포지션 상태를 유지하고 같은 청산 의도의 reconciliation을 계속하며, Position 수량이 0이 된 뒤에만 `CASE_*_SELL_FILLED` 또는 `FORCE_SELL_FINISHED`를 발생시킨다.

Context와 STM 상태가 잠시 어긋나는 것을 막기 위해 `CASE_*_POSITION_OPENED`, `CASE_*_SELL_FILLED`, `CASE_*_SELL_FINISHED`, `FORCE_SELL_FINISHED`를 포함한 Controller 생성 후속 EVENT는 이미 대기 중인 후속 시장 EVENT보다 먼저 처리한다. 재귀 호출은 사용하지 않고 현재 Action batch가 끝난 뒤 내부 queue의 다음 microstep에서 처리한다. 시간·시장 값 변화를 기다리는 EVENT는 `QueueEvent`가 아니라 `ScheduleReevaluation`을 사용한다.

`pending_exit_reason != None`인 동안 해당 포지션 Region의 일반 조건 검사 EVENT는 처리하지 않고 주문 결과와 retry EVENT만 처리한다. retry EVENT는 동일 청산 의도와 idempotency key를 사용한다.

0.4 외부 수동 매도와 새 실행 복구

앱 밖에서 발생한 매도는 아래 전략 Guard의 성공이나 `CASE_*_SELL_FILLED`로 추정하지 않는다. 실행 중에는 기존 외부 체결 reconciliation 차단을 유지한다. 새 process의 startup에서 `TradingController`가 실제 주문·fill·잔고를 반복 검증해 `EXTERNAL_MANUAL` 이력과 Position을 조정한다. 전량·부분 매도 및 수수료 잔량의 처리는 `ADR-007`을 따른다. 완료 후 상태는 `NOT_STARTED`이며, 매수 단계 진입이나 자동매매 시작 EVENT를 생성하지 않는다.

1. 상태 구성

1.1 최상위 상태

STATE ID	STM 표기	의미
LOGIC_ENABLED	Logic 가동 State	하단 BB 감시와 진입·매매 관리를 포함하는 최상위 복합 상태
LOWER_TOUCH_WATCH	하단 터치 감시 상태	현재가와 실시간 30분봉 하단 BB의 접촉을 감시하는 상태
TRADE_MANAGEMENT	전략 진입 및 매매 관리	포지션 영역, Case B 영역, Case C 영역을 병렬 실행하는 복합 상태
STOPPING	중지 처리 상태	포지션 보유 중지 요청 뒤 신규 진입을 차단하고 전량 매도 결과를 기다리는 상태
LOGIC_TERMINATED	Final State 기호	매매 중지 상태

1.2 ==병렬 진입·포지션 소유권== 영역 (Region_1)

상태 ID	STM 표기
NO_POSITION	포지션 미보유
CASE_B_POSITION_MANAGEMENT	Case B 포지션 관리 복합 상태
CASE_B_HOLDING	Lower_BB_30M_Pullback (Case_B) 포지션 보유
CASE_B_TREND_HOLD	강한 반등 상태 (TREND_HOLD)
CASE_B_CLOSED	Lower_BB_30M_Pullback (Case_B) 포지션 청산 완료
CASE_C_POSITION_MANAGEMENT	Case C 포지션 관리 복합 상태
CASE_C_HOLDING	Blade_Catching (Case_C) 포지션 보유
CASE_C_TP_TRAILING	익절권 진입 (TP_TRAILING)
CASE_C_CLOSED	Blade_Catching (Case_C) 포지션 청산 완료 및 %B >= 0.25 회복 대기
CASE_C_RECOVERY_SUCCEEDED	Case C 청산 후 realtime_pct_b >= 0.25 만족

1.3 Case B 신호 검사 Region

상태 ID	STM 표기
B_WAIT_TOUCH	Case B 터치봉 조건 대기
B_WAIT_SIGNAL	touch_candle_bbw < 0.02 만족 (WAIT_SIGNAL)
B_WAIT_PULLBACK	최초 확정 signal 만족 (WAIT_PULLBACK)
B_POSITION_OPEN_SIGNALLED	3시간 이내 && realtime_pct_b <= 0.30 만족 (POSITION_OPEN)
CASE_B_FINAL_STATE	Case B 신호 검사 종료

1.4 Case C 신호 검사 Region

상태 ID	STM 표기
C_WAIT_SETUP	Case C setup 조건 대기
C_SETUP	realtime_pct_b <= -0.15 && cci_30m_realtime <= -140 만족 (SETUP)
C_POSITION_OPEN_SIGNALLED	3분 회복 매수 조건 만족 (POSITION_OPEN)

상태 ID	STM 표기
CASE_C_FINAL_STATE	Case C 신호 검사 종료

1.5 공통 변수·판정 기준

변수·판정	정의
lower_band, upper_band	30분봉 close 기준 Bollinger Band(20기간, 표준편차 2)
realtime_pct_b	$(\text{realtime_price} - \text{lower_band}) / (\text{upper_band} - \text{lower_band})$
touch_candle	하단 BB를 최초로 터치한 30분봉. lower_band_at_touch와 Case B의 BBW는 이 봉의 값으로 고정한다.
confirmed_30m_close	현재 30분봉이 마감되어 증가와 지표값이 확정된 경우에만 True
confirmed_1m_close	현재 1분봉이 마감되어 close_1m이 확정된 판정 시점에만 True
signal_candle	하단 터치 뒤 Case B signal 조건을 최초로 모두 만족한 확정 30분봉
ema_slope_30m_close	확정 30분봉 close로 계산한 30분봉 EMA slope
realtime_ema_slope	현재가를 진행 중인 30분봉의 임시 close로 넣어 계산한 30분봉 EMA slope
current_close_ema_slope	확정 1분봉의 close_1m을 진행 중인 30분봉의 임시 close로 넣어 계산한 30분봉 EMA slope
cci_30m_realtime	30분봉 표준 CCI(20)를 실시간 현재가까지 반영해 계산한 값
holding_time	매수 체결시각부터 현재 판정시각까지의 경과 시간
position_owner	None, CASE_B, CASE_C 중 하나. terminal 주문 결과에서 실제 매수 체결 수량이 0보다 큼을 확인하고 Position에 반영한 뒤에 만 설정한다.
pending_strategy	주문 요청부터 terminal 결과 처리까지 예약된 CASE_B 또는 CASE_C. pending 주문이 없으면 None이다.
pending_order_side	pending 주문의 BUY 또는 SELL. pending 주문이 없으면 None이다.
pending_order_id	재조회와 중복 제출 방지에 사용하는 거래소/client 주문 식별자
pending_order_attempt_kind	매수 주문 시도 종류인 INITIAL 또는 RETRY
trading_phase	IDLE, ENTRY_ORDER_PENDING, EXIT_ORDER_PENDING, STOPPING, RECONCILIATION_REQUIRED, TERMINATED 중 하나
case_b_enabled, case_c_enabled	현재 하단 이벤트에서 각 Case의 signal 검사를 활성화할지 나타내는 플래그
case_b_entry_paused	Case C 보유 또는 비적극적 인계 구간에서 Case B 매수만 금지하는 플래그. signal 감시는 유지할 수 있다.
case_b_only_until_next_lower_touch	Case C를 재가동하지 않고 Case B만 감시하는 구간임을 나타내는 플래그
allow_new_case_c_setup	현재 하단 이벤트에서 새 Case C setup을 만들 수 있는지 나타내는 플래그
case_c_consumed_for_event	같은 하단 터치 이벤트에서 Case C가 종료된 뒤 재진입하지 못하게 하는 플래그
case_c_recovery_confirmed	Case C 종료 후 realtime_pct_b >= 0.25 회복을 확인했는지 나타내는 플래그
current_open_pct_b	Case C의 새 3분 회복 구간을 시작하는 시점의 realtime_pct_b 스냅샷
entry_pct_b	Case C 회복 진입선. timer_base_pct_b + 0.06으로 계산한다.
pending_exit_reason	매도 요청부터 terminal 결과 처리까지 보존하는 청산 사유
pending_return_state	매도 실패 시 유지할 Case B/C 포지션 관리 상태
case_b_exit_reason	Case B 청산 사유. EMERGENCY_STOP, STOP, TIME, TAKE_PROFIT, TREND_HOLD 중 하나이다.
case_c_exit_reason	Case C 청산 사유. TP_TRAIL, TP_FALLBACK, STOP, TIME 중 하나이다.
case_c_exit_pct_b	Case C 매도 체결 시점의 realtime_pct_b

2. Event-Action Table

모든 표에서 STM은 EVENT + Guard + 현재 상태로 행을 선택하고 다음 상태로 전이한다. TradingController 수행 Action은 선택된 행의 typed Action 요청을 Controller가 실제 수행하는 내용이다.

2.1 최상위 및 공통 Event-Action Table

ID	현재 상태	EVENT	가드·판정 조건	TradingController 수행 Action	다음 상태
G-01	시작	로직 시작	None	1) Trading Context를 초기화한다. 2) position_owner = None, pending 주문 값 None, trading_phase = IDLE을 적용한다.	LOWER_TOUCH_WATCH
G-02	LOWER_TOUCH_WATCH	하단 BB 접촉	realtime_price <= lower_band (같은 시점의 실시간 30분봉 BB)	1) lower_event_id, 터치 시각, touch_candle, 터치 순간의 실시간 30분봉 touch_candle_bbw 저장·고정, 2) case_b_enabled = (touch_candle_bbw <	TRADE

ID	현재 상태	EVENT	가드.판정 조건	TradingController 수행 Action	다음 상태
				0.02), 3) case_c_enabled = True, allow_new_case_c_setup = True, 4) case_c_consumed_for_event = False, case_c_recovery_confirmed = False, case_c_exit_reason = None, case_c_exit_pct_b = None, 5) case_b_entry_paused = False, case_b_only_until_next_lower_touch = False, 6) ACTIVATE_TRADE_MANAGEMENT EVENT 발생	
G-03	TRADE_MANAGEMENT	새 30분봉 하단 BB 접촉	position_owner == None && pending_order_id == None && current_30m_candle_id != touch_candle_id && realtime_price <= lower_band && (case_c_consumed_for_event == False OR case_c_recovery_confirmed == True)	1) 기존 하단 터치 이벤트를 종료한다. 2) 현재 30분봉을 touch_candle로 하여 새 lower_event_id를 생성한다. 3) G-02의 event-local Context 플래그를 재초기화한다. 4) ACTIVATE_TRADE_MANAGEMENT EVENT를 queue에 넣는다.	TRADE의 세 R에 재진입
G-04	TRADE_MANAGEMENT	TRADE_MANAGEMENT 완료	position_owner == None && Case B Region == CASE_B_FINAL_STATE && Case C Region == CASE_C_FINAL_STATE	1) 현재 하단 이벤트 종료, 2) 하단 터치 감시 재개	LOWE
G-05	임의의 무포지션 상태	매매 중지	position_owner == None && pending_order_id == None	pending 및 event-local Context를 정리하고 timer-구독-신규 event 수신을 종료한 뒤 trading_phase = TERMINATED로 변경한다.	LOGIC_
G-06	임의의 포지션 보유 상태	매매 중지[포지션 보유]	position_owner in {CASE_B, CASE_C} && pending_order_id == None	1) 신규 진입과 일반 조건 검사를 차단하고 trading_phase = STOPPING으로 변경한다. 2) ForceSellAll을 실행한다. 3) 정상 체결-Position-이력 반영 완료 시 position_owner = None으로 변경하고 FORCE_SELL_FINISHED EVENT를 queue에 넣는다. 4) 확정 실패 시 FORCE_SELL_FAILED EVENT를 queue에 넣고 상태 불명/부분 체결이면 같은 주문 ID를 우선 조화-조정한다.	STOPP
G-06P	LOGIC_ENABLED의 임의 상태	매매 중지[pending 주문 존재]	pending_order_id != None	1) 신규 전략 Action을 차단하고 trading_phase = STOPPING으로 변경한다. 2) pending 진입 주문은 취소-조화하고, pending 청산 주문은 terminal 결과까지 조화-조정한다. 3) 실제 fill을 Position-이력에 먼저 반영한다. 4) reconciliation 후 잔여 포지션이 있으면 전략 매도하고, 없으면 FORCE_SELL_FINISHED EVENT를 queue에 넣는다.	STOPP
G-06F	STOPPING	FORCE_SELL_FINISHED	position_owner == None && pending_order_id == None && 강제 매도 체결-저장 완료	1) pending 및 event-local Context를 정리한다. 2) timer-구독-신규 event 수신을 종료한다. 3) trading_phase = TERMINATED로 변경한다.	LOGIC_
G-06R	STOPPING	FORCE_SELL_FAILED	position_owner in {CASE_B, CASE_C} && 전략 매도가 terminal 미체결로 확정됨	retry/backoff 정책에 따라 동일 포지션의 전략 매도를 재시도한다. 상태 불명 또는 실제 fill 존재 시 신규 주문을 만들지 않고 reconciliation을 먼저 수행한다.	STOPP
G-07	TRADE_MANAGEMENT의 임의 상태	UPPER_BAND_TOUCHED	realtime_price >= upper_band	UpperBandPolicy.SAFE_TERMINATION을 적용한다. 1) pending_order_id != None이면 trading_phase = STOPPING, CancelPendingOrder(reason = UPPER_BAND_SAFE_TERMINATION), 같은 ID의 ReconcileOrder(stop_after_reconciliation = True)를 순서대로 요청하고 ForceSellAll은 요청하지 않는다. 2) pending이 없고 position_owner in {CASE_B, CASE_C}이면 trading_phase = STOPPING, 전략 평가 취소, ForceSellAll을 순서대로 요청하고 후속 결과를 G-06F/G-06R로 처리한다. 3) 둘 다 없으면 lower event와 Case B/C Context, pending 필드를 정리하고 trading_phase = TERMINATED, session 평가 취소, StopTradingRuntime(reason = UPPER_BAND_SAFE_TERMINATION)를 순서대로 요청한다.	branch branch LOGIC_

G-07 상단 BB 안전 종료 계약

G-07의 세 branch는 배타적이며 pending 주문 branch가 포지션 branch보다 우선한다. 따라서 pending 주문과 `position_owner`가 동시에 있어도 즉시 전량 매도하지 않고 취소·reconciliation으로 실제 fill과 잔여 수량을 먼저 확정한다. `stop_after_reconciliation = True`는 조정 결과에 잔여 포지션이 있으면 기존 STOP 계약의 전량 매도로 이어지고, 없으면 종료 후속 event를 만든다는 뜻이다.

이 정책은 상단 BB에서 새로운 매매 전략을 시작하지 않는다. 미구현 상단 상태 머신으로의 인계나 no-op 대신, 이 표에 이미 정의된 STOPPING, reconciliation, runtime 종료 계약을 재사용한다.

2.2 병렬 진입·포지션 소유권 Event-Action Table(Region 1)

ID	현재 상태	EVENT	가드	TradingController 수행 Action	다음 상태
O-01	포지션 영역 시작	초기 진입	None	<code>position_owner = None</code>	NO_POSITION
O-02	NO_POSITION	CASE_B_POSITION_OPENED	<code>position_owner == CASE_B</code> && <code>pending_order_id == None</code> && Case B 매수 체결 ·Position·이력 반영 완료	START_CASE_B_CONDITION_CHECK EVENT를 queue에 넣는다.	CASE_B_POSITION_MANAGEMENT
O-03	NO_POSITION	CASE_B_BUY_FAILED	<code>position_owner == None</code> && <code>attempt_kind == INITIAL</code> && Case B 매수 주문이 terminal 미체결로 확정됨	retry/backoff 정책에 따라 CASE_B_BUY_RETRY EVENT를 예약한다.	NO_POSITION
O-04	NO_POSITION	CASE_B_BUY_RETRY	<code>position_owner == None</code> && <code>pending_order_id == None</code> && <code>case_b_entry_paused == False</code> && signal의 3시간 유효 범위 안임	1) <code>pending_strategy = CASE_B</code> , <code>pending_order_side = BUY</code> , <code>pending_order_attempt_kind = RETRY</code> , <code>trading_phase = ENTRY_ORDER_PENDING</code> 으로 설정한다. 2) Case B 재매수 주문을 실행한다. 3) 정상 체결 후 Position·이력을 반영하고 <code>position_owner = CASE_B</code> , pending 값 해제 후 CASE_B_POSITION_OPENED EVENT를 queue에 넣는다. 4) terminal 미체결이면 pending 값 해제 후 CASE_B_BUY_FAILED(RETRY) EVENT를 queue에 넣는다.	NO_POSITION
O-05	NO_POSITION	CASE_B_BUY_FAILED	<code>position_owner == None</code> && <code>attempt_kind == RETRY</code> && Case B 재매수 주문이 terminal 미체결로 확정됨	retry/backoff 정책에 따라 CASE_B_BUY_RETRY EVENT를 다시 예약한다.	NO_POSITION
O-06	NO_POSITION	CASE_C_POSITION_OPENED	<code>position_owner == CASE_C</code> && <code>pending_order_id == None</code> && Case C 매수 체결 ·Position·이력 반영 완료	1) <code>case_b_entry_paused = True</code> 를 적용한다. 2) START_CASE_C_CONDITION_CHECK EVENT를 queue에 넣는다.	CASE_C_POSITION_MANAGEMENT
O-07	NO_POSITION	CASE_C_BUY_FAILED	<code>position_owner == None</code> && <code>attempt_kind == INITIAL</code> && Case C 매수 주문이 terminal 미체결로 확정됨	retry/backoff 정책에 따라 CASE_C_BUY_RETRY EVENT를 예약한다.	NO_POSITION
O-08	NO_POSITION	CASE_C_BUY_RETRY	<code>position_owner == None</code> && <code>pending_order_id == None</code> && <code>allow_new_case_c_setup == True</code> && <code>case_c_consumed_for_event == False</code>	1) <code>pending_strategy = CASE_C</code> , <code>pending_order_side = BUY</code> , <code>pending_order_attempt_kind = RETRY</code> , <code>trading_phase = ENTRY_ORDER_PENDING</code> 으로 설정한다. 2) Case C 재매수 주문을 실행한다. 3) 정상 체결 후 Position·이력을 반영하고 <code>position_owner = CASE_C</code> , pending 값 해제 후 CASE_C_POSITION_OPENED EVENT를 queue에 넣는다. 4) terminal 미체결이면 pending 값 해제 후 CASE_C_BUY_FAILED(RETRY) EVENT를 queue에 넣는다.	NO_POSITION
O-09	NO_POSITION	CASE_C_BUY_FAILED	<code>position_owner == None</code> && <code>attempt_kind == RETRY</code> && Case C 재매수 주문이 terminal 미체결로 확정됨	retry/backoff 정책에 따라 CASE_C_BUY_RETRY EVENT를 다시 예약한다.	NO_POSITION

동일 평가 주기에 `case_c_buy_signal`과 `case_b_buy_signal`이 함께 참이면 STM0이 Case C의 `SubmitOrder` Action만 선택한다. Controller는 Case C pending 주문 자리를 예약하고 Case B Action을 실행하지 않는다. `position_owner`는 어느 경우에도 매수 체결 전에는 설정하지 않는다.

2.2.1 CASE_B_POSITION_MANAGEMENT Event-Action Table

청산 Action을 처음 요청하는 Guard에는 `pending_order_id == None && pending_exit_reason == None`을 공통 적용한다. `pending_exit_reason`이 설정된 뒤에는 해당 주문의 결과 또는 `CASE_B_SELL_RETRY`만 처리한다.

ID	현재 상태	EVENT	Guard	TradingController 수행 Action	다음
PB-01	Initial Pseudo State	초기 진입	None	없음	CASE_B_HOLDING
PB-02	CASE_B_HOLDING	START_CASE_B_CONDITION_CHECK	<code>realtime_price > entry_price * 0.99 && NOT (confirmed_30m_close == True && ema_slope_30m_close < -0.08) && NOT (realtime_pct_b >= 0.60 5초 유지 && realtime_ema_slope <= 0.08) && NOT (realtime_pct_b >= 0.60 5초 유지 && realtime_ema_slope > 0.08 5초 유지) && holding_time < 6시간</code>	RETRY_CASE_B_CONDITION_CHECK EVENT 발생	CASE_B_HOLDING
PB-03	CASE_B_HOLDING	START_CASE_B_CONDITION_CHECK	PB-02의 부정, 즉 비상손절. 일반손절.Trend Hold 전환.일 반익절.시간청산 중 하나 이상이 참	우선순위에 따라 1) <code>realtime_price <= entry_price * 0.99</code> 이면 CASE_B_EMERGENCY_STOP, 2) <code>confirmed_30m_close == True && ema_slope_30m_close < -0.08</code> 이면 CASE_B_STOP, 3) <code>%B >= 0.60</code> 5초 및 <code>slope > 0.08</code> 5초이면 CASE_B_UPPER_TREND, 4) <code>%B >= 0.60</code> 5초 및 <code>slope <= 0.08</code> 이면 CASE_B_TAKE_PROFIT, 5) <code>holding_time >= 6시간</code> 이면 CASE_B_TIME_EXIT EVENT 발생	CASE_B_HOLDING
PB-04	CASE_B_HOLDING	RETRY_CASE_B_CONDITION_CHECK	PB-02와 동일	RETRY_CASE_B_CONDITION_CHECK EVENT 재발생	CASE_B_HOLDING
PB-05	CASE_B_HOLDING	RETRY_CASE_B_CONDITION_CHECK	PB-03과 동일	PB-03의 우선순위와 조건식을 그대로 적용하여 해당 EVENT 발생	CASE_B_HOLDING
PB-06	CASE_B_HOLDING	CASE_B_EMERGENCY_STOP(비상 손절)	<code>realtime_price <= entry_price * 0.99</code>	<code>pending_exit_reason = EMERGENCY_STOP, pending_return_state = CASE_B_HOLDING</code> 으로 저장하고 Case B 매도 주문을 실행한다. terminal 결과는 공통 주문 결과 계약에 따라 <code>CASE_B_SELL_FILLED</code> 또는 <code>CASE_B_SELL_FAILED</code> EVENT로 queue에 넣는다.	CASE_B_HOLDING
PB-07	CASE_B_HOLDING	CASE_B_SELL_FAILED	<code>pending_exit_reason == EMERGENCY_STOP && pending_return_state == CASE_B_HOLDING</code>	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.	CASE_B_HOLDING
PB-08	CASE_B_HOLDING	CASE_B_STOP(일반 손절)	<code>confirmed_30m_close == True && ema_slope_30m_close < -0.08</code>	<code>pending_exit_reason = STOP, pending_return_state = CASE_B_HOLDING</code> 으로 저장하고 Case B 매도 주문을 실행한다. terminal 결과는 <code>CASE_B_SELL_FILLED</code> 또는 <code>CASE_B_SELL_FAILED</code> EVENT로 queue에 넣는다.	CASE_B_HOLDING
PB-09	CASE_B_HOLDING	CASE_B_SELL_FAILED	<code>pending_exit_reason == STOP && pending_return_state == CASE_B_HOLDING</code>	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.	CASE_B_HOLDING

ID	현재 상태	EVENT	Guard	TradingController 수행 Action	다음
PB-10	CASE_B_HOLDING	CASE_B_TIME_EXIT(시간 청산)	holding_time >= 6시간	pending_exit_reason = TIME, pending_return_state = CASE_B_HOLDING으로 저장하고 Case B 매도 주문을 실행한다. terminal 결과는 CASE_B_SELL_FILLED 또는 CASE_B_SELL_FAILED EVENT로 queue에 넣는다.	CA!
PB-11	CASE_B_HOLDING	CASE_B_SELL_FAILED	pending_exit_reason == TIME && pending_return_state == CASE_B_HOLDING	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.	CA!
PB-12	CASE_B_HOLDING	CASE_B_TAKE_PROFIT(일반 익절)	realtime_pct_b >= 0.60 5초 유지 && realtime_ema_slope <= 0.08	pending_exit_reason = TAKE_PROFIT, pending_return_state = CASE_B_HOLDING으로 저장하고 Case B 매도 주문을 실행한다. terminal 결과는 CASE_B_SELL_FILLED 또는 CASE_B_SELL_FAILED EVENT로 queue에 넣는다.	CA!
PB-13	CASE_B_HOLDING	CASE_B_SELL_FAILED	pending_exit_reason == TAKE_PROFIT && pending_return_state == CASE_B_HOLDING	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.	CA!
PB-14	CASE_B_HOLDING	CASE_B_UPPER_TREND(강한 반등)	realtime_pct_b >= 0.60 5초 유지 && realtime_ema_slope > 0.08 5초 유지	CASE_B_TREND_HOLD_CONDITION_CHECK EVENT 발생	CA!
PB-15	CASE_B_TREND_HOLD	CASE_B_TREND_HOLD_CONDITION_CHECK	NOT (realtime_ema_slope <= 0.04 5초 유지) && NOT (realtime_pct_b < 0.60 5초 유지)	CASE_B_TREND_HOLD_CONDITION_CHECK EVENT 재발생	CA!
PB-16	CASE_B_TREND_HOLD	CASE_B_TREND_HOLD_CONDITION_CHECK	realtime_ema_slope <= 0.04 5초 유지 OR realtime_pct_b < 0.60 5초 유지	CASE_B_TREND_HOLD_SELL EVENT 발생	CA!
PB-17	CASE_B_TREND_HOLD	CASE_B_TREND_HOLD_SELL	realtime_ema_slope <= 0.04 5초 유지 OR realtime_pct_b < 0.60 5초 유지	pending_exit_reason = TREND_HOLD, pending_return_state = CASE_B_TREND_HOLD로 저장하고 Case B 매도 주문을 실행한다. terminal 결과는 CASE_B_SELL_FILLED 또는 CASE_B_SELL_FAILED EVENT로 queue에 넣는다.	CA!
PB-18	CASE_B_TREND_HOLD	CASE_B_SELL_FAILED	pending_exit_reason == TREND_HOLD && pending_return_state == CASE_B_TREND_HOLD	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.	CA!
PB-19	CASE_B_HOLDING	CASE_B_SELL_RETRY	pending_exit_reason in {EMERGENCY_STOP, STOP, TIME, TAKE_PROFIT} && pending_return_state == CASE_B_HOLDING && pending_order_id == None	같은 청산 사유와 idempotency key로 Case B 매도 주문을 실행한다. terminal 결과는 CASE_B_SELL_FILLED 또는 CASE_B_SELL_FAILED EVENT로 queue에 넣는다.	CA!
PB-20	CASE_B_HOLDING	CASE_B_SELL_FAILED	pending_exit_reason in {EMERGENCY_STOP, STOP, TIME, TAKE_PROFIT} && pending_return_state == CASE_B_HOLDING	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 다시 예약한다.	CA!
PB-21	CASE_B_TREND_HOLD	CASE_B_SELL_RETRY	pending_exit_reason == TREND_HOLD &&	같은 청산 사유와 idempotency key로 Case B 매도 주문을 실행한다. terminal 결과는	CA!

ID	현재 상태	EVENT	Guard	TradingController 수행 Action	다음
			<code>pending_return_state == CASE_B_TREND_HOLD && pending_order_id == None</code>	CASE_B_SELL_FILLED 또는 CASE_B_SELL_FAILED EVENT로 queue에 넣는다.	
PB-22	CASE_B_TREND_HOLD	CASE_B_SELL_FAILED	<code>pending_exit_reason == TREND_HOLD && pending_return_state == CASE_B_TREND_HOLD</code>	CASE_B_SELL_RETRY EVENT를 retry/backoff 정책에 따라 다시 예약한다.	CASE_B_SELL_RETRY
PB-23F	CASE_B_HOLDING 또는 CASE_B_TREND_HOLD	CASE_B_SELL_FILLED	<code>position_owner == None && case_b_exit_reason == pending_exit_reason && 매도 체결·Position-이력 반영 완료</code>	1) 주문 pending 값, <code>pending_exit_reason</code> , <code>pending_return_state</code> 를 해제하고 <code>trading_phase = IDLE</code> 로 변경한다. 2) CASE_B_SELL_FINISHED EVENT를 queue에 넣는다.	CASE_B_SELL_FINISHED
PB-23	CASE_B_CLOSED	CASE_B_SELL_FINISHED	<code>case_b_exit_reason in {STOP, EMERGENCY_STOP} && (realtime_price <= lower_band (같은 시점의 실시간 30분봉 BB))</code>	1) 실시간 접촉 순간의 30분봉을 새 <code>touch_candle</code> 로 하여 새 하단 이벤트를 생성한다. 2) <code>case_b_enabled = (touch_candle_bbw < 0.02)</code> , <code>case_c_enabled = True</code> , <code>allow_new_case_c_setup = True</code> 를 적용한다. 3) <code>case_c_consumed_for_event = False</code> , <code>case_c_recovery_confirmed = False</code> , <code>case_c_exit_reason = None</code> , <code>case_c_exit_pct_b = None</code> 으로 초기화한다. 4) <code>case_b_entry_paused = False</code> , <code>case_b_only_until_next_lower_touch = False</code> 를 적용한다. 5) ACTIVATE_TRADE_MANAGEMENT EVENT를 queue에 넣는다.	TRADING_CONTROLLER의 상태 재조
PB-24	CASE_B_CLOSED	CASE_B_SELL_FINISHED	NOT (PB-23의 Guard)	1) Case B runtime Context를 초기화한다. 2) 현재 하단 이벤트를 종료한다.	LOW

Case B 청산 판정 우선순위

- 비상 손절
- 확정 30분봉 일반 손절
- `realtime_pct_b >= 0.60` 5초 유지 시 `realtime_ema_slope > 0.08`의 5초 유지 여부로 Trend Hold/일반 익절 분기
- 일반 익절
- 시간 청산
- 공통 G-07의 `realtime_price >= upper_band` 상단 BB 안전 종료

`realtime_ema_slope`는 현재가를 임시 30분봉 close로 넣어 계산한다. 동일 평가 주기에 여러 조건이 참이면 위 순서대로 하나의 EVENT만 발생시킨다.

2.2.2 CASE_C_POSITION_MANAGEMENT Event-Action Table

청산 Action을 처음 요청하는 Guard에는 `pending_order_id == None && pending_exit_reason == None`을 공통 적용한다. `pending_exit_reason`이 설정된 뒤에는 해당 주문의 결과 또는 CASE_C_SELL_RETRY만 처리한다.

ID	현재 상태	EVENT	Guard	TradingController 수행 Action
PC-01	Initial Pseudo State	초기 진입	None	없음
PC-02	CASE_C_HOLDING	START_CASE_C_CONDITION_CHECK	<code>realtime_pct_b < 0.10 && NOT (realtime_ema_slope <= -0.55 3분 연속 유지) && holding_time < 60분</code>	RETRY_CASE_C_CONDITION_CHECK EVENT 발생
PC-03	CASE_C_HOLDING	START_CASE_C_CONDITION_CHECK	<code>realtime_pct_b >= 0.10 OR realtime_ema_slope <= -0.55 3분 연속 유지 OR holding_time >= 60분</code>	우선순위에 따라 1) <code>%B >= 0.10</code> 이면 CASE_C_ENTER_PROFIT_ZONE, 2) <code>sl <= -0.55</code> 3분 유지이면 CASE_C_STOP <code>holding_time >= 60분</code> 이면 CASE_C_TIME_EXIT EVENT 발생
PC-04	CASE_C_HOLDING	RETRY_CASE_C_CONDITION_CHECK	PC-02와 동일	RETRY_CASE_C_CONDITION_CHECK EVENT 재발생

ID	현재 상태	EVENT	Guard	TradingController 수행 Action
PC-05	CASE_C_HOLDING	RETRY_CASE_C_CONDITION_CHECK	PC-03과 동일	PC-03의 우선순위와 조건식을 그대로 적용하여 해당 EVENT 발생
PC-06	CASE_C_HOLDING	CASE_C_STOP(손절)	$realtime_ema_slope \leq -0.55$ 3분 연속 유지	$pending_exit_reason = STOP$, $pending_return_state = CASE_C_HOLDING$ 으로 저장하고 Case C 주문을 실행한다. terminal 결과는 공통 주문 계약에 따라 $CASE_C_SELL_FILLED$ 또는 $CASE_C_SELL_FAILED$ EVENT로 queue된다.
PC-07	CASE_C_HOLDING	CASE_C_SELL_FAILED	$pending_exit_reason == STOP$ && $pending_return_state == CASE_C_HOLDING$	CASE_C_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.
PC-08	CASE_C_HOLDING 또는 CASE_C_TP_TRAILING	CASE_C_TIME_EXIT(시간 청산)	$holding_time \geq 60$ 분	$pending_exit_reason = TIME$, $pending_return_state =$ 청산 시 상태로 저장하고 Case C 매도 주문을 실행한 terminal 결과는 $CASE_C_SELL_FILLED$ 또는 $CASE_C_SELL_FAILED$ EVENT로 queue된다.
PC-09	CASE_C_HOLDING 또는 CASE_C_TP_TRAILING	CASE_C_SELL_FAILED	$pending_exit_reason == TIME$ && 현재 상태가 $pending_return_state$ 와 같음	CASE_C_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.
PC-10	CASE_C_HOLDING	CASE_C_ENTER_PROFIT_ZONE(익절권 진입)	$realtime_pct_b \geq 0.10$	1) 익절권 진입 시점의 밴드로 $tp_price = lower_band + 0.10 * (upper_band - lower_band)$, 2) $previous_trail_ema_slope = ema_slope_30m(tp_price)$, 3) START_TP_TRAILING_CONDITION_CHECK EVENT 발생
PC-11	CASE_C_TP_TRAILING	START_TP_TRAILING_CONDITION_CHECK	$realtime_pct_b \geq 0.10$ && $holding_time < 60$ 분 && $confirmed_1m_close == False$	RETRY_TP_TRAILING_CONDITION_CHECK EVENT 발생
PC-12	CASE_C_TP_TRAILING	START_TP_TRAILING_CONDITION_CHECK	$realtime_pct_b < 0.10$ OR $confirmed_1m_close == True$ OR $holding_time \geq 60$ 분	우선순위에 따라 1) $\%B < 0.10$ 이면 $CASE_C_SELL_AT_TP_PRICE$, 2) 확정되면 $current_close_ema_slope = ema_slope_30m(close_1m)$ 계산 후 $current_close_ema_slope > previous_trail_ema_slope$ 이면 $CASE_C_EMA_INCREASEMENT$, 3) $current_close_ema_slope \leq previous_trail_ema_slope$ 이면 $CASE_C_EMA_DECREASEMENT$, 4) $holding_time \geq 60$ 분이면 $CASE_C_TIME_EXIT$ EVENT 발생
PC-13	CASE_C_TP_TRAILING	RETRY_TP_TRAILING_CONDITION_CHECK	PC-11과 동일	RETRY_TP_TRAILING_CONDITION_CHECK EVENT 재발생
PC-14	CASE_C_TP_TRAILING	RETRY_TP_TRAILING_CONDITION_CHECK	PC-12와 동일	PC-12의 우선순위와 조건식을 그대로 적용하여 해당 EVENT 발생
PC-15	CASE_C_TP_TRAILING	CASE_C_EMA_INCREASEMENT(30m EMA slope 증가)	$confirmed_1m_close == True$ && $current_close_ema_slope > previous_trail_ema_slope$	1) $previous_trail_ema_slope = current_close_ema_slope$, 2) RETRY_TP_TRAILING_CONDITION_CHECK EVENT 발생
PC-16	CASE_C_TP_TRAILING	CASE_C_SELL_AT_TP_PRICE(회복 강도 약화)	$realtime_pct_b < 0.10$	$pending_exit_reason = TP_FALL$, $pending_return_state = CASE_C_TP_TRAILING$ 로 저장하고 Case C 매도 주문을 실행한다. terminal 결과는 $CASE_C_SELL_FILLED$ 또는 $CASE_C_SELL_FAILED$ EVENT로 queue된다.

ID	현재 상태	EVENT	Guard	TradingController 수행 Action
PC-17	CASE_C_TP_TRAILING	CASE_C_SELL_FAILED	<code>pending_exit_reason == TP_FALLBACK</code> && <code>pending_return_state == CASE_C_TP_TRAILING</code>	CASE_C_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.
PC-18	CASE_C_TP_TRAILING	CASE_C_EMA_DECREASEMENT(30m EMA slope 비증가)	<code>confirmed_1m_close == True</code> && <code>current_close_ema_slope <= previous_trail_ema_slope</code>	<code>pending_exit_reason = TP_TRAIL</code> <code>pending_return_state = CASE_C_TP_TRAILING</code> 로 저장하고 확정 close 시점의 Case C 매도 주문을 실행한다 terminal 결과는 <code>CASE_C_SELL_FILLED</code> 또는 <code>CASE_C_SELL_FAILED</code> EVENT로 queue한다.
PC-19	CASE_C_TP_TRAILING	CASE_C_SELL_FAILED	<code>pending_exit_reason == TP_TRAIL</code> && <code>pending_return_state == CASE_C_TP_TRAILING</code>	CASE_C_SELL_RETRY EVENT를 retry/backoff 정책에 따라 예약한다.
PC-20	CASE_C_HOLDING	CASE_C_SELL_RETRY	<code>pending_return_state == CASE_C_HOLDING</code> && <code>pending_order_id == None</code>	같은 청산 사유와 idempotency key로 Case 매도 주문을 실행한다. terminal 결과는 <code>CASE_C_SELL_FILLED</code> 또는 <code>CASE_C_SELL_FAILED</code> EVENT로 queue한다.
PC-21	CASE_C_HOLDING	CASE_C_SELL_FAILED	<code>pending_return_state == CASE_C_HOLDING</code>	CASE_C_SELL_RETRY EVENT를 retry/backoff 정책에 따라 다시 예약한다.
PC-22	CASE_C_TP_TRAILING	CASE_C_SELL_RETRY	<code>pending_return_state == CASE_C_TP_TRAILING</code> && <code>pending_order_id == None</code>	같은 청산 사유와 idempotency key로 Case 매도 주문을 실행한다. terminal 결과는 <code>CASE_C_SELL_FILLED</code> 또는 <code>CASE_C_SELL_FAILED</code> EVENT로 queue한다.
PC-23	CASE_C_TP_TRAILING	CASE_C_SELL_FAILED	<code>pending_return_state == CASE_C_TP_TRAILING</code>	CASE_C_SELL_RETRY EVENT를 retry/backoff 정책에 따라 다시 예약한다.
PC-23F	CASE_C_HOLDING 또는 CASE_C_TP_TRAILING	CASE_C_SELL_FILLED	<code>position_owner == None</code> && <code>case_c_exit_reason == pending_exit_reason</code> && 매도 체결·Position-이력 반영 완료	1) 주문 pending 값, <code>pending_exit_reason</code> , <code>pending_return_state</code> 를 해제하고 <code>trading_phase = IDLE</code> 로 변경한다. 2) CASE_C_SELL_FINISHED EVENT를 queue에 넣는다.
PC-24	CASE_C_CLOSED	CASE_C_SELL_FINISHED	None	1) <code>case_c_consumed_for_event = True</code> , <code>allow_new_case_c_setup = False</code> 를 적용한다. 2) <code>case_b_entry_paused = True</code> 를 켜다. 3) CHECK_CASE_C_RECOVERY EVENT queue에 넣는다.
PC-25	CASE_C_CLOSED	CHECK_CASE_C_RECOVERY	<code>realtime_pct_b < 0.25</code>	CHECK_CASE_C_RECOVERY EVENT 발생
PC-26	CASE_C_CLOSED	CHECK_CASE_C_RECOVERY	<code>realtime_pct_b >= 0.25</code>	1) <code>case_c_recovery_confirmed = True</code> , 2) CHECK_CASE_B_HANDOFF EVENT 발생
PC-27	CASE_C_RECOVERY_SUCCEEDED	CHECK_CASE_B_HANDOFF	<code>case_c_exit_reason == TP_TRAIL</code> && <code>case_c_exit_pct_b < 0.40</code>	1) <code>case_b_entry_paused = False</code> , <code>case_b_only_until_next_lower = True</code> , 3) 같은 이벤트의 <code>allow_new_case_c_setup = False</code> 4) CASE_B_ACTIVE_RESUME EVENT 발생
PC-28	CASE_C_RECOVERY_SUCCEEDED	CHECK_CASE_B_HANDOFF	NOT (<code>case_c_exit_reason == TP_TRAIL</code> && <code>case_c_exit_pct_b < 0.40</code>)	1) <code>case_b_entry_paused = True</code> , 2) <code>case_b_only_until_next_lower = True</code> , 3) 같은 이벤트의 <code>allow_new_case_c_setup = False</code> 4) CASE_B_WAIT_ONLY EVENT 발생

Case C 청산 판정 우선순위

익절권 진입 전:

- `realtime_pct_b >= 0.10` 익절권 진입
- `realtime_ema_slope <= -0.55` 3분 연속 유지 손절

3. 60분 시간 청산

익절권 진입 후:

1. `realtime_pct_b < 0.10` TP_FALLBACK
2. 1분봉 마감 기준 `ema_slope_30m(close_1m)`와 `previous_trail_ema_slope` 비교
3. 60분 시간 청산

익절권 진입 후에는 Case C 손절 조건을 다시 적용하지 않는다. 1분봉 EMA가 아니라, 1분봉 close를 현재 30분봉의 임시 close로 넣어 계산한 30분봉 EMA slope를 사용한다.

2.3 Case B 신호 Event-Action Table(Region 2)

ID	현재 상태	EVENT	Guard	TradingConti
B-01	Initial Pseudo State	초기 진입	None	없음
B-02	B_WAIT_TOUCH	ACTIVATE_TRADE_MANAGEMENT	<code>touch_candle_bbw >= 0.02</code>	1) <code>case_b_er</code> runtime Conte
B-03	B_WAIT_TOUCH	ACTIVATE_TRADE_MANAGEMENT	<code>touch_candle_bbw < 0.02</code>	1) <code>case_b_er</code> False, 3) 확정
B-04	B_WAIT_SIGNAL	30M_CANDLE_CLOSED	<code>NOT (ema_slope_30m_close > -0.03 && pct_b_close > 0.25 && current_closed_candle.low >= min(previous_3_closed_candles.low))</code>	다음 30분봉 마
B-05	B_WAIT_SIGNAL	30M_CANDLE_CLOSED	<code>signal_created == False && ema_slope_30m_close > -0.03 && pct_b_close > 0.25 && current_closed_candle.low >= min(previous_3_closed_candles.low)</code>	1) 현재 확정봉을 <code>signal_crea</code> <code>candle_clo</code> START_CASE, EVENT 발생
B-06	B_WAIT_PULLBACK	START_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	<code>position_owner == None && pending_order_id == None && case_b_entry_paused == False && elapsed_time_from_signal <= 3시간 && realtime_pct_b <= 0.30</code>	1) <code>pending_s</code> <code>pending_or</code> <code>pending_or</code> <code>trading pha</code> 다. 2) Case B 이력을 반영하고 값 해제 후 CAS 에 넣는다. 4) te CASE_B_BUY 5) 상태 불명/부
B-07	B_WAIT_PULLBACK	START_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	<code>elapsed_time_from_signal <= 3시간 && (realtime_pct_b > 0.30 OR position_owner != None OR case_b_entry_paused == True)</code>	RETRY_CASE, EVENT 발생
B-08	B_WAIT_PULLBACK	START_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	<code>elapsed_time_from_signal > 3시간</code>	1) 해당 signal을 기화한다.
B-09	B_WAIT_PULLBACK	RETRY_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	<code>position_owner == None && pending_order_id == None && case_b_entry_paused == False && elapsed_time_from_signal <= 3시간 && realtime_pct_b <= 0.30</code>	B-06과 같은 C
B-10	B_WAIT_PULLBACK	RETRY_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	<code>elapsed_time_from_signal <= 3시간 && (realtime_pct_b > 0.30 OR position_owner != None OR case_b_entry_paused == True)</code>	RETRY_CASE, EVENT 재발생
B-11	B_WAIT_PULLBACK	RETRY_CASE_B_WAIT_PULLBACK_CONDITION_CHECK	<code>elapsed_time_from_signal > 3시간</code>	1) 해당 signal을 기화한다.
B-12	B_WAIT_SIGNAL	CASE_C_POSITION_OPENED	<code>position_owner == CASE_C</code>	<code>case_b_enti</code> 태와 변수는 유지
B-13	B_WAIT_PULLBACK	CASE_C_POSITION_OPENED	<code>position_owner == CASE_C</code>	<code>case_b_enti</code> 간 타이머는 유지
B-14	B_WAIT_SIGNAL	CASE_B_ACTIVE_RESUME	<code>case_c_recovery_confirmed == True && case_c_exit_reason == TP_TRAIL && case_c_exit_pct_b < 0.40</code>	<code>case_b_enti</code> 개

ID	현재 상태	EVENT	Guard	TradingController 수행 Action
B-15	B_WAIT_PULLBACK	CASE_B_ACTIVE_RESUME	<code>case_c_recovery_confirmed == True</code> <code>&& case_c_exit_reason == TP_TRAIL</code> <code>&& case_c_exit_pct_b < 0.40</code>	<code>case_b_entry_paused</code> 타이머를 유지한다
B-16	B_WAIT_SIGNAL	CASE_B_WAIT_ONLY	<code>NOT (case_c_exit_reason == TP_TRAIL</code> <code>&& case_c_exit_pct_b < 0.40)</code>	<code>case_b_entry_paused</code> 지 signal 판정
B-17	B_WAIT_PULLBACK	CASE_B_WAIT_ONLY	<code>NOT (case_c_exit_reason == TP_TRAIL</code> <code>&& case_c_exit_pct_b < 0.40)</code>	<code>case_b_entry_paused</code> 속 경과하며 신규
B-18	B_POSITION_OPEN_SIGNALLED	CASE_B_POSITION_OPENED	<code>position_owner == CASE_B</code>	없음
B-19	B_POSITION_OPEN_SIGNALLED	CASE_C_POSITION_OPENED	<code>position_owner == CASE_C</code>	1) 미체결 Case <code>case_b_entry_paused</code> <code>signal_time</code>

Case B 신호 판정 메모

- `touch_candle_bbw < 0.02`는 매수 시점 값이 아니라 하단 BB 터치봉 값이다.
- signal은 하단 터치 뒤 `ema_slope_30m_close > -0.03, pct_b_close > 0.25, signal_candle.low >= min(previous_3_closed_candles.low)`를 최초로 모두 만족한 확정 30분봉 하나만 인정한다.
- `previous_3_closed_candles`는 signal candle 바로 앞의 확정 30분봉 3개이며, 터치봉 기준 저점 비교가 아니다.
- 3시간은 `signal_time`부터 계산하며, 정확히 3시간인 시점은 진입 허용 범위에 포함하고 > 3시간이면 signal을 폐기한다.
- Case C 보유 중에도 Case B signal 상태는 유지할 수 있지만 `case_b_entry_paused == True`이면 Case B SubmitOrder Action을 만들지 않는다.
- B-06/B-09의 매수 Action 결과가 도착할 때까지 `B_POSITION_OPEN_SIGNALLED`를 유지한다. 실패 시 O-03/O-05가 retry를 예약하고, 정상 체결 시 동일 `CASE_B_POSITION_OPENED` EVENT를 O-02와 B-18이 각 Region에서 처리한다.

2.4 Case C 신호 Event-Action Table(Region 3)

ID	현재 상태	EVENT	Guard	TradingController 수행 Action
C-01	Initial Pseudo State	초기 진입	None	없음
C-02	C_WAIT_SETUP	ACTIVATE_TRADE_MANAGEMENT	<code>NOT (allow_new_case_c_setup == True && case_c_consumed_for_event == False && last_case_c_setup_candle_id != current_30m_candle_id && realtime_pct_b <= -0.15 && cci_30m_realtime <= -140)</code>	RETRY_C_WAIT_SETUP EVENT 발생
C-03	C_WAIT_SETUP	ACTIVATE_TRADE_MANAGEMENT	<code>allow_new_case_c_setup == True && case_c_consumed_for_event == False && last_case_c_setup_candle_id != current_30m_candle_id && realtime_pct_b <= -0.15 && cci_30m_realtime <= -140</code>	1) <code>last_case_c_setup_candle_id</code> , <code>current_30m_candle_id</code> , 2) <code>START_CASE_C_SETUP_CONDITION_CHECK</code> EVENT 발생
C-04	C_WAIT_SETUP	RETRY_C_WAIT_SETUP	C-02와 동일	RETRY_C_WAIT_SETUP EVENT 재발생
C-05	C_WAIT_SETUP	RETRY_C_WAIT_SETUP	C-03과 동일	1) <code>last_case_c_setup_candle_id</code> , <code>current_30m_candle_id</code> , 2) <code>START_CASE_C_SETUP_CONDITION_CHECK</code> EVENT 발생
C-06	C_WAIT_SETUP	CASE_B_POSITION_OPENED	<code>position_owner == CASE_B</code>	Case C 신규 진입 권한 종료
C-07	C_SETUP	START_CASE_C_SETUP_CONDITION_CHECK	None	1) <code>flush_low = None, flush_low_time = None, flush_high_time = None, timer_base_pct_b = None, timer_base_time = None, entry_base_time = None</code> , 3) <code>RETRY_CASE_C_SETUP_CONDITION_CHECK</code> EVENT 발생
C-08	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	<code>position_owner == None && realtime_pct_b >= 0.25</code>	1) 매수 없이 Case C setup 종료, 2) <code>case_c_consumed_for_event</code>

ID	현재 상태	EVENT	Guard	TradingController 수행 Action
				case_c_recovery_confirmed = True, allow_new_case_c_setup = False
C-09	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	flush_low == None && realtime_pct_b <= -0.25	1) flush_low = current_price, flush_low_pct_b = realtime_pct_b, flush_low_time = now, 2) timer_base_pct_b = flush_low_pct_b, timer_base_time = now, 3) entry_pct_b = timer_base_pct_b - 0.06, 4) RETRY_CASE_C_SETUP_CONDITION_CHECK EVENT 재발생
C-10	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	flush_low != None && current_price < flush_low	1) flush_low = current_price, flush_low_pct_b = realtime_pct_b, flush_low_time = now, 2) timer_base_pct_b = flush_low_pct_b, timer_base_time = now, 3) entry_pct_b = timer_base_pct_b - 0.06, 4) RETRY_CASE_C_SETUP_CONDITION_CHECK EVENT 재발생
C-11	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	flush_low != None && current_price >= flush_low && now - timer_base_time > 3분 && realtime_pct_b < 0.25	1) current_open_pct_b = realtime_pct_b, 2) timer_base_pct_b = current_open_pct_b, timer_base_time = now, 3) entry_pct_b = timer_base_pct_b - 0.06, 4) RETRY_CASE_C_SETUP_CONDITION_CHECK EVENT 재발생
C-12	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	position_owner == None && pending_order_id == None && flush_low != None && now - timer_base_time <= 3분 && realtime_pct_b >= entry_pct_b && entry_pct_b < -0.15 && realtime_pct_b < 0.25	1) 매수 판정 확정 시각을 저장한다. 2) pending_strategy = CASE_C, pending_order_side = BUY, pending_order_attempt_kind = INITIAL, trading_phase = ENTRY_ORDER_PENDING으로 설정한 Case C 매수 주문을 실행한다. 4) 정상 Position-이력을 반영하고 position_owner = CASE_C, pending 값 해제 후 CASE_C_POSITION_OPENED EVENT를 queue에 넣는다. 5) terminal 미체결이 값 해제 후 CASE_C_BUY_FAILED(IN EVENT)를 queue에 넣는다. 6) 상태 불일치는 같은 주문 ID를 조회·조정한다.
C-13	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	flush_low != None && now - timer_base_time <= 3분 && realtime_pct_b >= entry_pct_b && entry_pct_b >= -0.15 && realtime_pct_b < 0.25	1) 추격매수하지 않음, 2) flush 추적과 (entry_pct_b - 0.06) setup 유지, 3) RETRY_CASE_C_SETUP_CONDITION_CHECK EVENT 재발생
C-14	C_SETUP	RETRY_CASE_C_SETUP_CONDITION_CHECK	realtime_pct_b < 0.25 && ((flush_low == None && realtime_pct_b > -0.25) OR (flush_low != None && current_price >= flush_low && now - timer_base_time <= 3분 && realtime_pct_b < entry_pct_b))	RETRY_CASE_C_SETUP_CONDITION_CHECK EVENT 재발생
C-15	C_SETUP	CASE_B_POSITION_OPENED	position_owner == CASE_B	Case C 신규 진입 권한 종료; setup 기간만 보존
C-16	C_POSITION_OPEN_SIGNALLED	CASE_C_POSITION_OPENED	position_owner == CASE_C	없음
C-17	C_POSITION_OPEN_SIGNALLED	CASE_B_POSITION_OPENED	position_owner == CASE_B	1) 미체결 Case C 매수·재시도 취소, 2) 신규 진입 권한 종료

Case C 신호 판정 메모

- SETUP은 `realtime_pct_b <= -0.15 && cci_30m_realtime <= -140`일 때 시작하고, 첫 flush는 `realtime_pct_b <= -0.25`에서 확정한다.
- `entry_pct_b = timer_base_pct_b + 0.06`이며, 3분 이내 `&& realtime_pct_b >= entry_pct_b && entry_pct_b < -0.15`를 모두 만족해야 매수한다.
- `entry_pct_b >= -0.15`이면 회복 폭을 만족해도 매수하지 않으며, Case와 flush 추적은 유지한다.
- 더 낮은 가격이 발생한 경우 C-10의 flush 갱신을 C-11의 3분 타이머 재시작보다 먼저 처리한다.
- C_SETUP의 판정 순서는 1) 매수 전 `%B >= 0.25` 종료, 2) 최초 flush 또는 더 낮은 저점 갱신, 3) 3분 초과 시 타이머 재시작, 4) 3분 이내 회복 매수/비매수, 5) 감시 재시도 순이다.
- 한 30분봉에서 Case C setup은 한 번만 인정한다. 매수 전 `realtime_pct_b >= 0.25`이면 해당 setup을 종료한다.
- Case C 매도 후에는 `realtime_pct_b >= 0.25` 회복 전까지 같은 이벤트의 Case C 재진입을 허용하지 않는다. 포지션 보유 중에는 `%B >= 0.25`를 초기화 조건으로 사용하지 않는다.
- C-12의 매수 Action 결과가 도착할 때까지 `C_POSITION_OPEN_SIGNALLED`를 유지한다. 실패 시 O-07/O-09가 retry를 예약하고, 정상 체결 시 동일 `CASE_C_POSITION_OPENED` EVENT를 O-06과 C-16이 각 Region에서 처리한다.